🛠️ Divisão de Tarefas - Backend FarmaGestor

✅ Premissas

- Pessoa 1 vai cuidar da estrutura base e da configuração inicial.
- Pessoas 2, 3 e 4 focam em funcionalidades específicas.
- Comunicação constante é fundamental, principalmente sobre modelos e permissões.
- Todo mundo precisa estar familiarizado com Git, branches e PRs.

🔄 Divisão de Tarefas

👤 Pessoa 1 – Setup Inicial, Autenticação e Integração

Setup Inicial:

- Criar o projeto: `django-admin startproject FarmaGestor_core`
- Criar o app: `python manage.py startapp gestor_api`
- Configurar o `settings.py` com:
  - `gestor_api`, `rest_framework`, `corsheaders`, `AUTH_USER_MODEL`
- Configurar ambiente virtual (venv) e passar as instruções pro time
- Criar e configurar o `.gitignore` com: `venv/`, `db.sqlite3`, `__pycache__/`
- Configurar as rotas iniciais no `urls.py` principal com prefixo `/api/`

Usuários:

- Criar o modelo `Usuario` com `NivelAcesso` (herdando de `AbstractUser`)
- Rodar as migrations e criar o superusuário

Autenticação:

- Criar a view `CustomAuthToken` para login
- Criar rota `/api/login/`
- Ativar `rest_framework.authtoken` no projeto
- Ajustar `DEFAULT_AUTHENTICATION_CLASSES` no `settings.py`

Gerenciamento de Usuários (Admin):

- Criar `UsuarioSerializer` e `UsuarioViewSet`
- Permissão: apenas `IsAdminUser`
- Rota: `/api/usuarios/`

Extra:

- Ser ponto de integração do grupo
- Ajudar com dúvidas e revisar PRs

💊 Pessoa 2 – Módulo de Medicamentos (CRUD)

Modelo:

- Criar o modelo `Medicamento` no `models.py`
- Rodar `makemigrations` e `migrate`

Serializers:

- Criar `MedicamentoSerializer`

Views:

- Criar `MedicamentoViewSet`
- Permissões:
  - `create`, `update`, `destroy`: apenas Operacional/Admin
  - `list`, `retrieve`: qualquer usuário autenticado

URLs:

- Adicionar as rotas no `router` do `urls.py`

Testes (opcional):

- Criar testes dos endpoints de medicamentos

📦 Pessoa 3 – Módulo de Movimentação de Estoque

Modelo:

- Criar `Movimentacao`, com `ForeignKey` para `Medicamento` e `Usuario`
- Rodar `makemigrations` e `migrate`

Serializers:

- Criar `MovimentacaoSerializer`

Views:

- Criar `MovimentacaoViewSet`
- Implementar lógica de atualização de estoque (`perform_create`)
- Permissões:
  - `create` , `list` , `retrieve` : Operacional/Admin
  - `update` , `destroy` : apenas Admin

URLs:

- Adicionar as rotas no `router`

Testes (opcional):

- Criar testes para os endpoints de movimentações

🚨 Pessoa 4 – Alertas e Relatórios

Views:

- Criar função `alertas_validade` (validade dos medicamentos)
- Criar função `relatorios_movimentacao` com lógica de filtro

URLs:

- Adicionar rotas diretas no `urls.py` :
  - `/api/alertas/validade/`
  - `/api/relatorios/`

Melhorias (extras):

- Melhorar a lógica de alertas (validade mais próxima, etc.)
- Melhorar os filtros e dados dos relatórios

Testes (opcional):

- Testes dos endpoints de alertas e relatórios

Documentação (opcional):

- Começar com docstrings e, se der, usar Swagger/OpenAPI com `drf-spectacular`

🔁 Fluxo de Trabalho Recomendado

1. Pessoa 1 faz o setup inicial e sobe pro GitHub (`main` ou `master`)

2. O restante:
    - Clona o repositório
    - Cria sua venv e instala as dependências com `pip install -r requirements.txt`
3. Cada um cria sua própria branch:
    - `feature/medicamentos`
    - `feature/movimentacoes`
    - `feature/alertas-relatorios`
4. Desenvolve sua parte de forma isolada
5. Ao finalizar e testar localmente, cria um **Pull Request**
6. Pessoa 1 (ou quem for designado) revisa e faz o merge
7. Todo mundo mantém a branch atualizada com `git pull origin main`